

Inventory Management System for B2B SaaS

StockFlow – Case Study Submission

Overview

"StockFlow" is a B2B inventory management platform aimed at helping small businesses track product quantities across multiple warehouses and manage supplier relationships. This document contains the take-home case study response, including code review, schema design, and API implementation.

Part 1: Code Review & Debugging

Original Flask Code

```
@app.route('/api/products', methods=['POST'])
def create_product():
    data = request.json
    product = Product(
        name=data['name'],
        sku=data['sku'],
        price=data['price'],
        warehouse_id=data['warehouse_id']
    )
    db.session.add(product)
    db.session.commit()
    inventory = Inventory(
```

```
        product_id=product.id,  
        warehouse_id=data['warehouse_id'],  
        quantity=data['initial_quantity']  
    )  
  
    db.session.add(inventory)  
  
    db.session.commit()  
  
    return {"message": "Product created", "product_id":  
product.id}
```

Issues Identified

- **Incorrect Inventory Relationship:** Product shouldn't have a direct `warehouse\_id` field. Inventory is a separate many-to-many mapping.
- **No SKU Uniqueness Check:** Doesn't prevent duplicate SKUs.
- **Missing Field Validation:** No error handling or field validation.
- **No Optional Handling:** `initial\_quantity` may be missing.
- **No Transaction Handling:** If inventory insert fails, product is still committed.

Impact in Production

- Products wrongly tied to a single warehouse.
- Inventory might not be tracked correctly.
- Duplicated SKUs can cause business confusion.
- Partial inserts may cause data inconsistencies.

Fixed Version (Node.js , Express)

```
const createProduct = async (req, res) => {  
    let connection;  
  
    try {
```

```
    const { name, sku, price, warehouse_id,
initial_quantity } = req.body;

    connection = await getDbConnection();
    await connection.beginTransaction();
    const [existingSku] = await connection.execute(
        'SELECT id FROM products WHERE sku = ?',
        [sku]
    );
    if (existingSku.length > 0) {
        await connection.rollback();
        return res.status(409).json({ error: 'SKU
already exists' });
    }
    const [result] = await connection.execute(
        'INSERT INTO products (name, sku, price,
warehouse_id, created_at) VALUES (?, ?, ?, ?, NOW())',
        [name, sku, price, warehouse_id]
    );
    const productId = result.insertId;
    if (initial_quantity !== undefined &&
initial_quantity >= 0) {
        await connection.execute(
```

```
        'INSERT INTO inventory (product_id,
warehouse_id, quantity, created_at) VALUES (?, ?, ?,
NOW())', [productId, warehouse_id, initial_quantity]

        );

    }
    await connection.commit();

    res.status(201).json({
        message: 'Product created successfully',
        product: { id: productId,
                    name,
                    sku,
                    price
                }
    });
} catch (error) {
    if (connection) await connection.rollback();
    console.error('Error creating product:', error);
    res.status(500).json({ error: 'Failed to create
product' });
} finally {
    if (connection) await connection.end();
}
};
```

Part 2: Database Schema Design

Entities and Relationships

- **Company**
 - Fields: id, name, email
 - One-to-many with Warehouse and Supplier
- **Warehouse**
 - Fields: id, name, companyId
 - Belongs to one Company
 - Many-to-many with Product via Inventory
- **Product**
 - Fields: id, name, sku, price, productType, lowStockThreshold, isActive
 - Many-to-many with Warehouse (Inventory)
 - Many-to-many with Supplier (ProductSupplier)
 - Self-referential many-to-many via ProductBundle
- **Inventory (Pivot Table)**
 - Fields: id, productId, warehouseId, quantity
 - Composite unique index on productId + warehouseId
- **Supplier**
 - Fields: id, name, contactEmail, companyId
 - Many-to-many with Product via ProductSupplier
- **ProductSupplier (Pivot Table)**
 - Fields: productId, supplierId
- **SalesHistory**
 - Fields: productId, warehouseId, quantitySold, saleDate, created_at
 - Tracks sales per product per warehouse
- **InventoryHistory (Audit Table)**
 - Fields: id, productId, warehouseId, quantityChange, changeType, reason, changedBy, changedAt

- Tracks inventory changes (adjustments, receipts, etc.)
- **ProductBundle (Pivot Table)**
 - Fields: parentProductId, childProductId, quantity
 - Used for kits and composite products

Integrity and Normalization

- Referential integrity via foreign keys
- Junction tables implement many-to-many
- Data validation via Sequelize (e.g., isEmail, min: 0)
- Indexing improves performance and avoids duplicates
- InventoryHistory supports full audit trail
- ProductBundle enables advanced catalog logic

Scalability Features

- Modular schema, extendable for future features
- Multi-tenancy via Company-Warehouse structure
- Product-level low stock thresholds for alerts
- InventoryHistory meets compliance needs
- Bundles allow for flexible catalog management

Part 3: API Implementation – Low Stock Alerts

Endpoint

GET /api/companies/:company_id/alerts/low-stock

Controller Logic (alertController.js)

```
const { QueryTypes } = require('sequelize');
const db = require('../models');
const getLowStockAlerts = async (req, res) => {
  try {
```

```

    const { company_id } = req.params;

    // Verify company exists

    const company = await
db.Company.findByPk(company_id);

    if (!company) {

        return res.status(404).json({ error: 'Company
not found' });

    }

    // query to get low stock alerts

    const alertsQuery = `

        SELECT

            p.id as product_id,

            p.name as product_name,

            p.sku,

            w.id as warehouse_id,

            w.name as warehouse_name,

            i.quantity as current_stock,

            p.lowStockThreshold as threshold,

            CASE

                WHEN sales.avg_daily_sales > 0

                    THEN FLOOR(i.quantity /
sales.avg_daily_sales)

                ELSE NULL

```

```

        END as days_until_stockout,
        s.id as supplier_id,
        s.name as supplier_name,
        s.contactEmail as supplier_contact_email
FROM products p
JOIN inventory i ON p.id = i.productId
JOIN warehouses w ON i.warehouseId = w.id
LEFT JOIN product_suppliers ps ON p.id =
ps.productId
LEFT JOIN suppliers s ON ps.supplierId = s.id
LEFT JOIN (
    SELECT
        productId,
        warehouseId,
        AVG(quantitySold) as avg_daily_sales
    FROM sales_history
    WHERE created_at >= DATE_SUB(NOW(),
INTERVAL 30 DAY)
    GROUP BY productId, warehouseId
    HAVING AVG(quantitySold) > 0
) sales ON p.id = sales.productId AND w.id =
sales.warehouseId
WHERE w.companyId = :companyId

```

```

        AND i.quantity <= p.lowStockThreshold
        AND p.isActive = 1
        AND (sales.avg_daily_sales > 0 OR
sales.avg_daily_sales IS NULL)
    ORDER BY (i.quantity / p.lowStockThreshold)
ASC, p.name`;

    // Run the query

    const alerts = await
db.sequelize.query(alertsQuery, {
        replacements: { companyId: company_id },
        type: QueryTypes.SELECT
    });

    // Format response

    const formattedAlerts = alerts.map(alert => ({
        product_id: alert.product_id,
        product_name: alert.product_name,
        sku: alert.sku,
        warehouse_id: alert.warehouse_id,
        warehouse_name: alert.warehouse_name,
        current_stock: alert.current_stock,
        threshold: alert.threshold,
        days_until_stockout:
alert.days_until_stockout,

```

```

        supplier: alert.supplier_id
        ? {
            id: alert.supplier_id,
            name: alert.supplier_name,
            contact_email:
alert.supplier_contact_email
        }
        : null
    )));

    res.json({
        alerts: formattedAlerts,
        total_alerts: formattedAlerts.length
    });

} catch (error) {
    console.error('Error fetching low stock alerts:',
error);

    res.status(500).json({ error: 'Internal server
error' });

}

};

module.exports = { getLowStockAlerts };

```

Edge Cases Handled

- Company not found
- No sales data
- Products with no suppliers
- Division by zero handled avoided via NULL

Assumptions Made

- Sales history is tracked daily.
- One supplier per product used in alert (first match)
- Inventory quantity is always positive integer.
- `lowStockThreshold` is always set on product level.

Stock Alert Endpoint – Postman Preview

